

Fundamentos del software

UNIVERSIDAD ESTATAL DE MILAGRO
FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE TECNOLOGÍAS DE LA INFORMACIÓN EN MODALIDAD
EN LÍNEA

TEMA:

Fundamentos del software

AUTOR:

Ángel Fernando Calero Maldonado

ASIGNATURA:

Fundamentos de Tecnologías de Información

DOCENTE:

Díaz Sandoya Eduardo Luis

FECHA DE ENTREGA:

21 de mayo de 2026

PERIODO:

abril 2026 a julio 2026

MILAGRO – ECUADOR

Índice

1	Introducción	4
2	Clasificación del software	4
2.1	Definición de software	4
2.2	Software de sistema	5
2.3	Software de aplicación	5
2.4	Software de programación	5
2.5	Clasificación según licencia	5
2.6	Tabla comparativa de clasificación del software	6
3	Calidad del software	7
3.1	Concepto de calidad del software	7
3.2	Norma ISO/IEC 25010	7
3.3	Evaluación de calidad de software conocido	7
3.4	Análisis de la evaluación	9
4	Lenguajes de alto y bajo nivel	9
4.1	Lenguajes de alto nivel	9
4.2	Ventajas y desventajas de los lenguajes de alto nivel	9
4.3	Lenguajes de bajo nivel	9
4.4	Ventajas y desventajas de los lenguajes de bajo nivel	10
4.5	Comparación entre lenguajes de alto y bajo nivel	10
5	Ejemplos prácticos	10
5.1	Ejemplo en Python	10
5.2	Ejemplo en pseudocódigo	11
5.3	Comparación con ensamblador	11
6	Importancia del software en la actualidad	11
7	Conclusiones	12

8	Referencias	13
----------	------------------------------	-----------

1. Introducción

El software constituye uno de los elementos más importantes dentro de los sistemas de cómputo, debido a que permite que el hardware pueda ejecutar instrucciones, procesar información y responder a las necesidades de los usuarios. En la actualidad, casi todas las actividades académicas, laborales, administrativas y personales dependen de algún tipo de software, desde los sistemas operativos que controlan una computadora hasta las aplicaciones móviles que facilitan la comunicación diaria.

El presente informe tiene como finalidad analizar los fundamentos del software desde tres aspectos principales: su clasificación, su calidad y los lenguajes utilizados para su desarrollo. Para ello, se estudian las categorías de software de sistema, software de aplicación y software de programación, así como los tipos de licencia más utilizados en el entorno tecnológico. Además, se realiza una evaluación de calidad de diferentes herramientas conocidas mediante criterios relacionados con la norma ISO/IEC 25010. Finalmente, se explican las diferencias entre lenguajes de alto y bajo nivel, incorporando ejemplos prácticos que permiten comprender mejor su funcionamiento.

Comprender estos temas es fundamental para la formación del ingeniero en Tecnologías de la Información, ya que permite reconocer cómo se construyen, evalúan y utilizan las soluciones informáticas dentro de un contexto real.

2. Clasificación del software

2.1 Definición de software

El software puede definirse como el conjunto de programas, instrucciones, datos y procedimientos que permiten que un sistema informático funcione correctamente. A diferencia del hardware, que corresponde a los componentes físicos de una computadora, el software es intangible y se encarga de indicar al equipo qué tareas debe realizar. Según Sommerville (2016), el software no solo está formado por programas, sino también por documentación, configuraciones y datos asociados que permiten su correcta operación.

2.2 Software de sistema

El software de sistema administra los recursos físicos del computador y permite la comunicación entre el usuario, las aplicaciones y el hardware. Su función principal es controlar procesos internos como la memoria, los archivos, los dispositivos de entrada y salida, la seguridad y la ejecución de programas.

Entre los ejemplos más representativos se encuentran Microsoft Windows, GNU/Linux y macOS. Estos sistemas operativos permiten que el usuario interactúe con la computadora de forma organizada y eficiente.

2.3 Software de aplicación

El software de aplicación está diseñado para realizar tareas específicas orientadas al usuario final. Permite desarrollar actividades como redactar documentos, navegar por Internet, editar imágenes, comunicarse o gestionar información.

Ejemplos comunes de software de aplicación son Microsoft Word, Google Chrome, WhatsApp, Zoom y Google Drive. Estas herramientas se utilizan ampliamente en contextos educativos, laborales y personales.

2.4 Software de programación

El software de programación está dirigido a los desarrolladores y permite crear, probar, depurar y mantener otros programas. Incluye lenguajes de programación, compiladores, intérpretes, editores de código y entornos de desarrollo integrado.

Algunos ejemplos son Visual Studio Code, PyCharm, NetBeans, GCC y Python IDLE.

2.5 Clasificación según licencia

El software también puede organizarse según su modelo de licencia. El software libre permite usar, estudiar, modificar y distribuir el programa respetando las libertades establecidas por su licencia. El software propietario restringe el acceso al código fuente y normalmente exige una licencia de uso. El software de código abierto permite consultar y modificar el código fuente

según las condiciones de su licencia. Finalmente, el software en la nube funciona mediante servicios alojados en Internet.

2.6 Tabla comparativa de clasificación del software

Categoría	Función principal	Ejemplos
Software de sistema	Administra los recursos del computador y permite la interacción entre hardware, usuario y aplicaciones.	Windows, GNU/Linux, macOS.
Software de aplicación	Permite realizar tareas específicas para el usuario final.	Microsoft Word, Google Chrome, WhatsApp.
Software de programación	Permite crear, probar y mantener otros programas informáticos.	Visual Studio Code, PyCharm, GCC.
Software libre	Puede usarse, estudiarse, modificarse y distribuirse bajo ciertas libertades.	LibreOffice, GIMP, GNU/Linux.
Software propietario	Su uso está restringido por una licencia y normalmente no permite acceder al código fuente.	Microsoft Office, Adobe Photoshop, Windows.
Software de código abierto	Permite revisar y modificar el código fuente según las condiciones de su licencia.	Mozilla Firefox, VLC Media Player, Apache OpenOffice.
Software en la nube	Funciona mediante servicios alojados en Internet.	Google Drive, Microsoft 365, Canva.

Tabla 1. Clasificación general del software con ejemplos representativos.

3. Calidad del software

3.1 Concepto de calidad del software

La calidad del software se refiere al grado en que un sistema, aplicación o programa cumple con los requisitos establecidos y satisface las necesidades del usuario. Un software de calidad no solo debe funcionar correctamente, sino que también debe ser seguro, fácil de usar, eficiente, confiable y adaptable a diferentes contextos.

3.2 Norma ISO/IEC 25010

La norma ISO/IEC 25010 forma parte del modelo SQuaRE y establece características para evaluar la calidad de productos de software. Este modelo considera atributos como adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.

Para este informe se consideran especialmente los siguientes atributos: funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad.

3.3 Evaluación de calidad de software conocido

Software	Atributo evaluado	Fortalezas	Debilidades
Microsoft Word	Funcionalidad y usabilidad	Permite crear documentos académicos, aplicar estilos, insertar tablas, revisar ortografía y exportar archivos.	Algunas funciones avanzadas pueden resultar complejas para usuarios nuevos.
WhatsApp	Fiabilidad y usabilidad	Facilita mensajes, llamadas, videollamadas y envío de archivos.	Depende de conexión a Internet y requiere protección adecuada de la cuenta.
Zoom	Eficiencia y compatibilidad	Permite reuniones virtuales, clases en línea, grabaciones y pantalla compartida.	Puede consumir muchos recursos de red cuando la conexión es baja.
Google Drive	Portabilidad y disponibilidad	Permite almacenar archivos en la nube y acceder desde distintos dispositivos.	Requiere Internet para aprovechar todas sus funciones.
Visual Studio Code	Mantenibilidad y funcionalidad	Ofrece extensiones, depuración, integración con Git y soporte para múltiples lenguajes.	El exceso de extensiones puede afectar el rendimiento.
Google Chrome	Eficiencia y compatibilidad	Permite navegar en la web, ejecutar aplicaciones en línea y sincronizar datos.	Puede consumir mucha memoria RAM con varias pestañas abiertas.

Tabla 2. Evaluación de calidad de software conocido según criterios relacionados con ISO/IEC 25010.

3.4 Análisis de la evaluación

La evaluación muestra que cada software posee fortalezas relacionadas con el propósito para el cual fue diseñado. Microsoft Word destaca en la creación de documentos; WhatsApp en la comunicación inmediata; Zoom en las reuniones virtuales; Google Drive en el almacenamiento en la nube; Visual Studio Code en el desarrollo de software; y Google Chrome en la navegación web.

Sin embargo, también se identifican debilidades importantes. Algunas herramientas requieren conexión permanente a Internet, otras consumen muchos recursos del computador y algunas pueden presentar dificultades de seguridad si el usuario no aplica buenas prácticas.

4. Lenguajes de alto y bajo nivel

4.1 Lenguajes de alto nivel

Los lenguajes de alto nivel son aquellos que utilizan una sintaxis cercana al lenguaje humano y facilitan la escritura de programas. Su principal ventaja es que permiten al programador concentrarse en la lógica del problema sin preocuparse demasiado por los detalles internos del hardware.

Entre los lenguajes de alto nivel más conocidos se encuentran Python, Java, JavaScript, C# y PHP.

4.2 Ventajas y desventajas de los lenguajes de alto nivel

Los lenguajes de alto nivel tienen como ventaja principal su facilidad de aprendizaje, lectura y mantenimiento. Además, permiten desarrollar aplicaciones de forma más rápida. Sin embargo, pueden ser menos eficientes que los lenguajes de bajo nivel porque requieren compiladores o intérpretes.

4.3 Lenguajes de bajo nivel

Los lenguajes de bajo nivel son aquellos que se encuentran más cerca del lenguaje máquina y del funcionamiento interno del procesador. Entre ellos se encuentran el lenguaje en-

samblador y el lenguaje máquina.

4.4 Ventajas y desventajas de los lenguajes de bajo nivel

Los lenguajes de bajo nivel ofrecen mayor control sobre el hardware y pueden generar programas muy eficientes. No obstante, su escritura es más compleja, menos intuitiva y más propensa a errores.

4.5 Comparación entre lenguajes de alto y bajo nivel

Aspecto	Lenguaje de alto nivel	Lenguaje de bajo nivel
Cercanía al usuario	Más cercano al lenguaje humano.	Más cercano al lenguaje de la máquina.
Facilidad de aprendizaje	Generalmente más fácil de aprender.	Requiere mayor conocimiento técnico del hardware.
Eficiencia	Puede ser menos eficiente por la traducción del código.	Puede ser más eficiente porque controla directamente recursos.
Portabilidad	Mayor posibilidad de ejecutarse en distintas plataformas.	Depende más de la arquitectura del procesador.
Ejemplos	Python, Java, JavaScript.	Ensamblador y lenguaje máquina.

Tabla 3. Comparación entre lenguajes de alto y bajo nivel.

5. Ejemplos prácticos

5.1 Ejemplo en Python

Un ejemplo sencillo de lenguaje de alto nivel en Python consiste en imprimir un mensaje en pantalla.

Código en Python

```
print("Hola, bienvenido a Fundamentos del Software")
```

En este caso, la instrucción `print()` permite mostrar un mensaje directamente en pantalla.

5.2 Ejemplo en pseudocódigo

El mismo ejemplo puede representarse en pseudocódigo de la siguiente manera:

Pseudocódigo
Algoritmo Mensaje Escribir "Hola, bienvenido a Fundamentos del Software" FinAlgoritmo

5.3 Comparación con ensamblador

En lenguaje ensamblador, una instrucción equivalente puede variar según la arquitectura del procesador y el sistema operativo. Una representación simplificada podría expresarse así:

Ejemplo simplificado en ensamblador
<pre>section .data mensaje db "Hola", 0 section .text global _start</pre>

La comparación evidencia que los lenguajes de alto nivel son más comprensibles y prácticos para la mayoría de usuarios y programadores. En cambio, el ensamblador requiere conocer la estructura interna del procesador y las instrucciones específicas que utiliza la máquina.

6. Importancia del software en la actualidad

El software es fundamental en la sociedad actual porque se encuentra presente en la educación, la salud, la banca, el comercio, la comunicación y la industria. Plataformas educativas, aplicaciones de mensajería, sistemas de gestión empresarial, servicios en la nube y herramientas de programación demuestran que el software ya no es un recurso aislado, sino una base esencial para el funcionamiento de la vida moderna.

En el ámbito universitario, el software permite acceder a aulas virtuales, entregar tareas, realizar investigaciones, comunicarse con docentes y desarrollar habilidades técnicas.

7. Conclusiones

El software es un componente esencial de todo sistema informático, ya que permite que el hardware pueda ejecutar tareas útiles para los usuarios. Su clasificación en software de sistema, aplicación y programación permite comprender mejor su función dentro del entorno tecnológico.

La calidad del software debe evaluarse considerando diferentes atributos, como funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad y portabilidad. Estos criterios ayudan a identificar si una herramienta cumple adecuadamente con las necesidades del usuario.

Los lenguajes de alto nivel facilitan el desarrollo de programas gracias a su sintaxis comprensible y cercana al lenguaje humano. Por otro lado, los lenguajes de bajo nivel permiten un mayor control del hardware, aunque requieren conocimientos técnicos más profundos.

Finalmente, el estudio de los fundamentos del software es importante para la formación profesional en Tecnologías de la Información, porque proporciona una base sólida para analizar, utilizar y desarrollar soluciones informáticas de manera responsable, eficiente y segura.

8. Referencias

- Free Software Foundation. (2024). *What is free software?* <https://www.gnu.org/philosophy/free-sw.html>
- ISO/IEC. (2011). *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.
- Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de información gerencial* (16.a ed.). Pearson Educación.
- Open Source Initiative. (2024). *The open source definition*. <https://opensource.org/osd>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.
- W3Schools. (2024). *Python print() function*. https://www.w3schools.com/python/ref_func_print.asp